Windows VM Migration Deep Technical Guide

Win10 / Win11 / Server — Zero-Touch Driver Installation

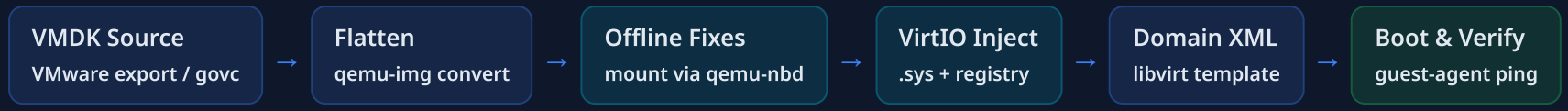
VirtIO Injection | Registry Editing | Firstboot Scripts | UEFI + BIOS | Apache 2.0

github.com/ssahani/hyper2kvm

April 2026 — v0.3.0

1. Migration Pipeline

End-to-end flow from VMware VMDK to a running KVM guest with full VirtIO drivers.



Conversion Stage

- Accept VMDK, OVA, VHD, or raw disk images
- Flatten snapshots into single QCOW2 file
- Optional compression for storage savings
- Checksum verification (SHA256) for integrity
- Sparse file support preserves thin provisioning

Offline Fix Stage

- Mount NTFS partitions via qemu-nbd + ntfs-3g
- Auto-detect Windows version from registry hive
- Inject VirtIO storage, network, balloon drivers
- Create firstboot service for PnP driver binding
- Disable VMware Tools services in registry

Deploy Stage

- Generate libvirt domain XML (Q35 + PCIe)
- Auto-detect UEFI vs BIOS firmware
- VirtIO disk bus + VirtIO NIC model
- SPICE or VNC graphics (auto-detected)
- `virsh define` + optional `virsh start`

Verification Stage

- Boot test with SATA fallback disk for safety
- Guest agent ping confirms driver installation
- Offline NBD remount verifies .sys file presence
- Production XML generated with VirtIO disk bus
- Optional KubeVirt manifest for Kubernetes deploy

Key principle: Every modification happens offline via qemu-nbd. The VM never needs to boot with broken drivers. VirtIO storage is registered as Start=0 (boot-start) so Windows loads it before any other disk driver.

2. VirtIO Driver Injection

Six drivers copied to System32\drivers with full registry service entries and PnP staging.

Driver	File	Purpose	Start Type	PCI Hardware IDs
viostor	viostor.sys	Block storage (boot-critical)	0 (Boot)	VEN_1AF4&DEV_1001, VEN_1AF4&DEV_1042
vioscsi	vioscsi.sys	SCSI controller	0 (Boot)	VEN_1AF4&DEV_1004, VEN_1AF4&DEV_1048
NetKVM	netkvm.sys	Network adapter	2 (Auto)	VEN_1AF4&DEV_1000, VEN_1AF4&DEV_1041
Balloon	balloon.sys	Dynamic memory	3 (Demand)	VEN_1AF4&DEV_1002, VEN_1AF4&DEV_1045
vioserial	vioser.sys	Serial port / guest agent	3 (Demand)	VEN_1AF4&DEV_1003, VEN_1AF4&DEV_1043
viorng	viorng.sys	Hardware RNG	3 (Demand)	VEN_1AF4&DEV_1005, VEN_1AF4&DEV_1044

CriticalDeviceDatabase

- Maps PCI vendor/device IDs to driver service names
- Windows checks this before loading any boot driver
- Both transitional (DEV_1001) and modern (DEV_1042) IDs registered
- ClassGUID set to SCSIAdapter for storage drivers

Services Registration

- ImagePath points to `system32\drivers\viostor.sys`
- Group = "SCSI miniport" for storage drivers
- ErrorControl = 1 (normal) to prevent boot failure loops
- Type = 1 (kernel driver) for all VirtIO services

INF/CAT Staging

- INF files copied to `Windows\INF\` as OEM entries
- SYS+INF staged in `DriverStore\FileRepository\`
- CAT catalog files staged for signature verification
- PnP auto-discovers staged drivers on first boot

viostor.sys (boot-critical) | netkvm.sys (network) | balloon.sys (memory) | vioser.sys (serial) | viorng.sys (entropy) | All registered offline via hivex

3. Firstboot Architecture

rhsrvany.exe service wrapper with multi-method fallback for reliable first-boot script execution.

rhsrvany.exe Service Wrapper

- Runs batch/PowerShell scripts as a Windows service
- Compatible with virt-v2v rhsrvany.exe format
- Service registered in SYSTEM hive: `Services\h2kfirstboot`
- Parameters\CommandLine points to staged script
- Parameters\PWD sets working directory
- Start=2 (auto-start) ensures execution at boot

Run Key Fallback (Win11 Edge Builds)

- Some Win11 edge builds delay service startup
- RunOnce registry key as fallback mechanism
- `SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce`
- Batch file triggers pnputil driver installation
- OOBE bypass keys set: BypassNRO, SkipMachineOOBE
- Auto-login enabled to reach desktop without prompts

Firstboot Script Pipeline

Service Starts rhsrvany.exe → **VirtIO ISO pnputil** all driver folders → **Custom PNP** C:\hyper2kvm-drivers → **Network Apply** PowerShell script → **Guest Agent MSI** qemu-ga install → **Self-Cleanup** remove service

Per-Service Scripts

- Each script is independent (no overwrite)
- VirtIO install script staged separately
- Network apply script runs standalone
- Custom driver script isolated from VirtIO
- Failure in one does not block others

Self-Cleanup

- Service deletes itself after successful run
- RunOnce key auto-removed by Windows
- Staged scripts removed from guest
- Auto-login reverted to normal behavior
- No hyper2kvm artifacts remain post-migration

Logging

- All output to `C:\hyper2kvm\firstboot.log`
- pnputil results captured per driver
- Network apply log at `apply-network.log`
- Exit codes recorded for troubleshooting
- Serial console output on COM1

Dual-method reliability: rhsrvany service executes before user login. RunOnce key provides a fallback at user login. Both methods are staged, so even if one path fails, the other completes driver installation.

4. Registry Editing

Offline registry manipulation via hivex for SYSTEM and SOFTWARE hives.

SYSTEM Hive Operations

- Service creation: viostor, vioscsi, netkvm, balloon, vioserial, viorng
- CriticalDeviceDatabase entries for PCI hardware IDs
- VMware service disablement (VMTools, vm3dservice, etc.)
- ControlSet001 targeted (CurrentControlSet not available offline)
- Start type: 0=Boot, 2=Auto, 3=Demand, 4=Disabled

SOFTWARE Hive Operations

- RunOnce key for firstboot batch script
- Auto-login: DefaultUserName, DefaultPassword, AutoAdminLogon
- OOBE bypass: BypassNRO, SkipMachineOOBE, SkipUserOOBE
- DevicePath update to include VirtIO driver store path
- VMware uninstall entries cleaned from registry

hivex Python API

```
import hivex

# Open SYSTEM hive for read-write
h = hivex.Hivex("/mnt/Windows/System32/config/SYSTEM", write=True)

# Navigate to Services key
root = h.root()
ccs = h.node_get_child(root, "ControlSet001")
services = h.node_get_child(ccs, "Services")

# Create viostor service entry
viostor = h.node_add_child(services, "viostor")
h.node_set_value(viostor, {"key": "Type", "t": 4, "value": struct.pack("<I", 1)})
h.node_set_value(viostor, {"key": "Start", "t": 4, "value": struct.pack("<I", 0)})
h.node_set_value(viostor, {"key": "ImagePath", "t": 1,
    "value": "system32\\drivers\\viostor.sys\0".encode("utf-16-le")})

h.commit(None)  # Write changes back to hive file
```

Path Conventions

- ImagePath uses literal `system32\drivers\` (relative to SystemRoot)
- No `$env:SystemRoot` expansion in registry values
- Literal `C:\` paths only for staged files outside System32
- DriverStore paths use full `C:\Windows\System32\DriverStore\`
- UTF-16LE encoding required for all REG_SZ values

VMware Cleanup

- 9 VMware services set to Start=4 (Disabled)
- VMware Tools uninstall keys removed
- vm3dservice, VGAuthService, vmvss disabled
- vmci, vsock kernel drivers disabled
- Prevents VMware driver conflicts with VirtIO

5. Auto-Detect & Compatibility

Automatic detection of host capabilities and guest requirements for seamless deployment.

Video Model Detection

- Default: QXL for SPICE, virtio-gpu for VNC
- Auto-detect: SPICE if libspice-server.so present
- Fallback: VNC with virtio-gpu (works everywhere)
- RHEL/AlmaLinux: typically VNC (no SPICE)
- Fedora: SPICE + QXL available

OVMF / Firmware Detection

- Scan standard paths: `/usr/share/OVMF/`
- Fedora: `/usr/share/edk2/ovmf/`
- RHEL: `/usr/share/OVMF/OVMF_CODE.secboot.fd`
- Secure Boot firmware preferred when available
- BIOS fallback for legacy MBR guests

VirtIO ISO Discovery

- Check: `/usr/share/virtio-win/virtio-win.iso`
- Check: `/var/lib/hyper2kvm/virtio-win.iso`
- deploy-remote.sh auto-downloads if missing
- Version matched to guest OS (w10, w11, 2k22)
- Architecture: amd64 selected for 64-bit guests

virsh define Auto-Fix

Domain XML Corrections

- QEMU binary path: `/usr/bin/qemu-system-x86_64` (Fedora) vs `/usr/libexec/qemu-kvm` (RHEL)
- Machine type: q35 with PCIe topology
- CPU mode: host-passthrough for full KVM acceleration
- Memory ballooning enabled with VirtIO balloon device
- Serial console on pty for headless access

Graphics Configuration

- SPICE: listen on localhost, auto-port, agent channel
- VNC: listen on localhost, auto-port, websocket
- Tablet input device for proper mouse tracking
- USB controller (qemu-xhci) for USB passthrough
- Sound device (ich9) for Windows audio support

QEMU binary auto-detect | SPICE/VNC auto-select | OVMF firmware scan | VirtIO ISO discovery | Q35 + PCIe topology | host-passthrough CPU

6. Tested Platforms

Verified Windows guest and Linux host combinations with production results.

Guest OS	Firmware	Boot	Drivers	Network	Notes
Win10 Pro 20H2	BIOS + UEFI	✓ Pass	✓ All 6	✓ DHCP/Static	Most common migration target
Win11 Pro 22H2	UEFI + SecureBoot	✓ Pass	✓ All 6	✓ DHCP/Static	OOBE bypass required
Win11 Edge Build	UEFI + SecureBoot	✓ Pass	✓ All 6	✓ DHCP	RunOnce fallback used
Server 2019	BIOS + UEFI	✓ Pass	✓ All 6	✓ Static	Core + Desktop Experience
Server 2022	UEFI	✓ Pass	✓ All 6	✓ Static	Azure Stack HCI source

Host Environments

Host OS	QEMU Path	Graphics	VirtIO ISO	Notes
AlmaLinux 9 (Worldstream)	/usr/libexec/qemu-kvm	VNC	✓ Auto	Bare-metal production server
Fedora 43	/usr/bin/qemu-system-x86_64	SPICE	✓ Auto	Development workstation
RHEL 8.8	/usr/libexec/qemu-kvm	VNC	✓ Auto	Enterprise production
Ubuntu 24.04	/usr/bin/qemu-system-x86_64	SPICE	✓ Manual	virtio-win from repo

Zero-Touch Windows Migration

Offline VirtIO injection + registry editing + firstboot scripts + auto-detection = fully automated migration.

No BSOD. No manual driver install. No IP reconfiguration. Works across Win10, Win11, and Server.

5 Windows versions tested | 4 host distros verified | UEFI + BIOS support | Secure Boot compatible | Zero BSOD incidents